

# RBF-GNN: RATIONAL BASIS FUNCTIONS FOR PSEUDO-COORDINATE BASED GRAPH CONVOLUTIONS

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

We propose RBF-GNN, a new pseudo-coordinate based graph neural network architecture that takes into account euclidean, spherical or angular coordinates and uses them to induce a powerful spatial inductive bias. Similar in architecture to SplineCNN, we improve upon the latter by replacing the less efficient sparse-activation based B-splines whose number grows exponentially with dimension by rational Padé basis functions. For effective training we propose a spline-subspace initialization and a variance-preserving weight rescaling. Experimentally, we evaluate on a number of popular neural network architectures that use SplineCNNs. We replace only the SplineCNNs with RBF-GNN. We achieve improved results, including on semantic keypoint matching, shape matching, event based camera computer vision tasks. We will make our implementation publicly available upon acceptance of the paper.

## 1 INTRODUCTION

Many graph convolutions for geometric data define their kernel as a continuous function of pseudo-coordinates attached to the edges, and they differ mainly in how this function is represented. SplineCNN (Fey et al., 2018) expands it in a fixed tensor-product basis of B-splines, MoNet (Monti et al., 2017) in a mixture of learnable Gaussians, and more recent continuous-kernel operators (Ma et al., 2024) in a small multilayer perceptron of the pseudo-coordinates. This choice determines which kernels a layer can represent and how many parameters it requires.

SplineCNN has been adopted in a range of later architectures. Its basis functions are compactly supported B-splines on a regular grid over the unit cube. This gives the kernel an explicit spatial locality and makes the evaluation time independent of the kernel size, because only the few basis functions whose support contains the pseudo-coordinate of an edge have to be evaluated.

The same grid, however, also determines the number of parameters. With  $k$  basis functions per axis in  $D$  pseudo-coordinate dimensions, the grid contains  $K = k^D$  basis functions, and the layer has one weight matrix for each of them. For a fixed resolution per axis, the number of weight matrices therefore grows exponentially with  $D$ , independently of the complexity of the kernel that has to be learned. Because of the compact support, each edge moreover involves only a small subset of these weight matrices, so that each weight matrix is updated by only a fraction of the edges.

In this work, we replace the B-spline basis by  $K$  learnable rational functions, so that  $K$  becomes a hyperparameter that is independent of  $D$ . Each basis function is a ratio of two polynomials in the safe Padé form  $B = P(t)/(1 + |Q(t)|)$  (Molina et al., 2020), whose denominator is at least one and therefore never vanishes, and both polynomials are expressed in the Chebyshev basis for better numerical conditioning. Unlike the B-splines, these functions are not compactly supported, so every edge involves all  $K$  weight matrices. Since rational functions with small random coefficients are nearly flat, we initialize the basis from the B-splines it replaces. When  $K = k^D$ , we fit the B-splines directly. Otherwise, we fit their leading principal components and rescale the kernel weights so that the initial message variance matches that of the B-spline layer.

In our experiments, the main benefit of the rational basis is parameter efficiency, and its extent depends on the size of the B-spline grid that is replaced rather than on the pseudo-coordinate dimension alone. For large grids, a rational basis with few functions matches the accuracy of the full grid with substantially fewer parameters. For small grids, it does not reduce the number of parameters

consistently but improves accuracy. The accuracy gains are moderate and vary across benchmarks. They are smaller in the stronger of the two keypoint matching pipelines, and on the six-dimensional part segmentation benchmark the B-spline grid remains slightly more accurate. A learned basis of the same size produced by a multilayer perceptron is less accurate than the rational basis in most comparisons, which indicates that the improvements are not due to learning the basis alone.

Our contributions are as follows:

- We remove the coupling between the size of a layer’s weights and the dimension of its coordinate space, and we establish what a learnable basis needs in order to take a fixed one’s place without loss
- We evaluate the substitution across eight benchmarks whose pseudo-coordinate dimension ranges from two to six, spanning keypoint matching on PascalVOC (Everingham et al., 2010) and SPair-71k (Min et al., 2019) in two published matching architectures (Fey et al., 2020; Pourhadi & Swoboda, 2026), shape correspondence on FAUST (Bogo et al., 2014), event-camera recognition on N-Caltech101 (Orchard et al., 2015) and N-Cars (Sironi et al., 2018), superpixel classification (Dwivedi et al., 2023) and point-cloud part segmentation (Yi et al., 2016). On that evidence we recommend a rational basis wherever a parameter budget binds and the spline grid would be wide.

## 2 RELATED WORK

**Geometry-aware graph convolutions.** Message-passing neural networks provide the general neighbourhood-aggregation view (Gilmer et al., 2017), while spatial operators additionally condition messages on edge geometry. ECC generates filters from edge attributes (Simonovsky & Komodakis, 2017); MoNet applies Gaussian mixtures to pseudo-coordinates (Monti et al., 2017); and FeaStNet learns feature-dependent assignments to weight matrices (Verma et al., 2018). Continuous filters have also been generated by neural networks for molecules (Schütt et al., 2017), point clouds (Wang et al., 2018; Wu et al., 2019), and general graphs equipped with positional encodings (Ma et al., 2024). Related point-cloud operators use Taylor-polynomial filters (Xu et al., 2018), learnable kernel points (Thomas et al., 2019), geometry-conditioned weight banks (Xu et al., 2021), or relative-position attention (Zhao et al., 2021). SplineCNN instead expands a continuous kernel in compactly supported tensor-product B-splines (Fey et al., 2018), an operator subsequently used for graph matching (Fey et al., 2020) and event streams (Schaefer et al., 2022). RBF-GNN is closest to SplineCNN: it preserves its message-passing rule and matrix bank, but replaces the fixed grid basis by learnable non-separable rational functions whose number is independent of  $k^D$ .

**Learned continuous and rational bases.** Learnable functional parameterisations include spline time–frequency atoms (Balestriero et al., 2018), neural continuous kernels (Romero et al., 2022b;a), and coordinate networks based on Fourier features, periodic activations, or multiplicative filters (Tancik et al., 2020; Sitzmann et al., 2020; Sahu et al., 2021). Rational functions have been studied as trainable activations (Molina et al., 2020) and as efficient smooth approximators (Boullé et al., 2020). More recently, KAN places learned B-spline functions on network edges (Liu et al., 2025), while KAT replaces those splines by rational functions to improve efficiency and scalability (Yang & Wang, 2025). Rational-activation and KAN-family models modify neuron or feature-mixing nonlinearities, while neural coordinate models learn an entire continuous signal or kernel. In contrast, our rational functions are multivariate mixing coefficients of a matrix-valued graph kernel and are initialised to preserve both the SplineCNN kernel subspace and its output variance.

## 3 METHOD

RBF-GNN is a graph convolution whose continuous kernel is spanned by learnable rational basis functions of pseudo-coordinates. We first set up pseudo-coordinates and the continuous-kernel convolution operator, then introduce the rational basis in safe Padé form. A longer remark relates the construction to SplineCNN (Fey et al., 2018), which uses the same operator with a fixed B-spline

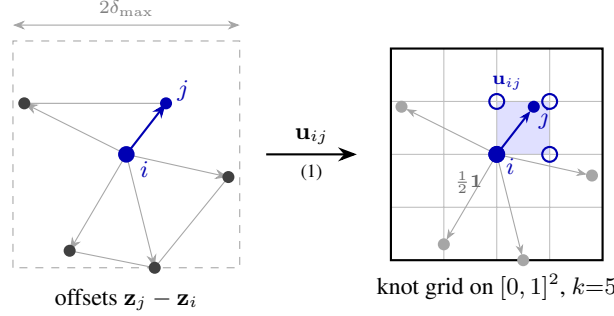


Figure 1: Pseudo-coordinates ( $D=2$ ). **Left:** the convolution at node  $i$  sees each neighbour  $j$  only through its spatial offset  $\mathbf{z}_j - \mathbf{z}_i$  (arrows); the dashed box marks the largest offset magnitude  $\delta_{\max}$  occurring in the graph. **Right:** the shift-and-scale map (1) sends the dashed box to the unit square, so the neighbourhood on the left reappears as a scaled copy: node  $i$  (zero offset) lands at the centre  $\frac{1}{2}\mathbf{1}$ , each neighbour at its pseudo-coordinate —  $j$  at  $\mathbf{u}_{ij}$  (blue) — and the largest offset touches the boundary (bottom dot). The B-spline kernel is defined once on this square via the  $k \times k$  knot grid: at  $\mathbf{u}_{ij}$  only the  $2^D=4$  hats anchored at the circled corners of the containing cell (shaded) are non-zero, so only 4 of the  $K=25$  weight matrices  $\Theta_p$  are used — and updated — for this edge.

basis and for which RBF-GNN is a drop-in replacement. Finally, we assemble the full network and describe the initialization that lets it train as reliably as the spline model it can replace.

**Graphs and pseudo-coordinates.** We consider graphs  $G = (V, E)$  embedded in a coordinate space: every node  $i$  carries features  $\mathbf{x}_i \in \mathbb{R}^{C_{\text{in}}}$  and a spatial position  $\mathbf{z}_i \in \mathbb{R}^D$  (2-D image keypoints, vertices of a 3-D mesh, ...). The convolution below depends on *relative* positions. Because the kernel will be parameterised by basis functions living on a *fixed, bounded* domain, the offsets are affinely rescaled into the unit cube: with  $\delta_{\max}$  the largest absolute offset component occurring in the graph, the *pseudo-coordinate* of edge  $(j, i)$  is

$$\mathbf{u}_{ij} = \frac{\mathbf{z}_j - \mathbf{z}_i}{2\delta_{\max}} + \frac{1}{2}\mathbf{1} \in [0, 1]^D \quad (1)$$

The zero offset lands at the centre  $\frac{1}{2}\mathbf{1}$ , a neighbour to the left/below has  $u_d < \frac{1}{2}$  in that dimension, and the largest offset touches the boundary. See Figure 1 for an illustration of the pseudo-coordinate construction. These pseudo-coordinates are the only way geometry enters the model. Polar or spherical encodings can be handled analogously.

**Continuous convolution kernels.** An image convolution assigns a separate weight matrix to each of the finitely many pixel offsets. On an irregular graph the offsets  $\mathbf{u}_{ij}$  vary continuously, so we instead learn a *kernel function*  $g: [0, 1]^D \rightarrow \mathbb{R}^{C_{\text{in}} \times C_{\text{out}}}$  and evaluate it at the observed pseudo-coordinates. The kernel is parameterised as a linear combination  $g(\mathbf{u}) = \sum_{p=1}^K B_p(\mathbf{u}) \Theta_p$  of  $K$  scalar basis functions  $B_p: [0, 1]^D \rightarrow \mathbb{R}$  with learnable weight matrices  $\Theta_p \in \mathbb{R}^{C_{\text{in}} \times C_{\text{out}}}$ , and a layer computes

$$\mathbf{x}'_i = \Theta_{\text{root}} \mathbf{x}_i + \square_{j \in \mathcal{N}(i)} \left( \sum_{p=1}^K B_p(\mathbf{u}_{ij}) \Theta_p \right) \mathbf{x}_j + \mathbf{b}, \quad (2)$$

with a separate root weight for the node itself, a bias, and aggregation  $\square \in \{\text{mean}, \text{add}\}$  over the neighbourhood. This operator was introduced by SplineCNN (Fey et al., 2018); the difference lies entirely in the choice of the basis functions  $B_p$ , which are fixed B-splines there (see the remark below) and learnable rational functions here.

**Rational safe Padé basis functions.** In RBF-GNN the scalar basis functions  $B_p: [0, 1]^D \rightarrow \mathbb{R}$  of (2) are learnable rational functions. Each  $B_p$  is a quotient of two polynomials whose coefficients are network parameters. We build up the functional form in one dimension first. A *polynomial* of degree

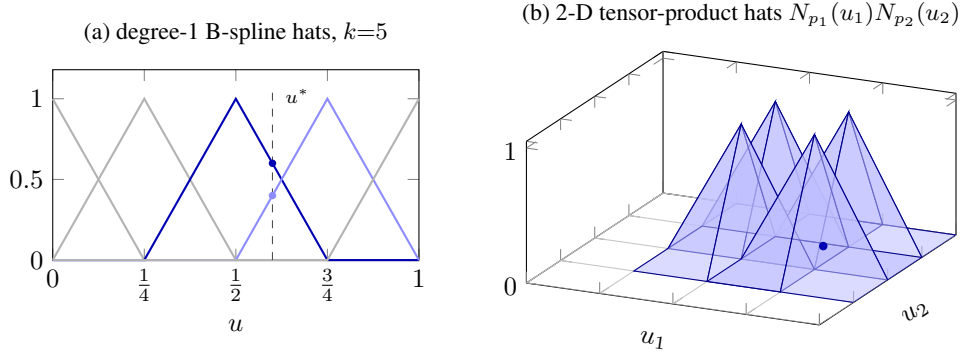


Figure 2: The B-spline basis. **(a)** The five open degree-1 B-splines on  $[0, 1]$ ; at any coordinate  $u^*$  (dashed) only two are non-zero (dots), so in  $D$  dimensions only  $2^D$  of the  $k^D$  tensor products activate. **(b)** 3-D view of the 2-D basis: each  $B_{\mathbf{p}}$  is a pyramid  $N_{p_1}(u_1)N_{p_2}(u_2)$  centred on a knot pair. Shown are the four pyramids anchored at the corners of the shaded cell of Figure 1; they are the only ones of the  $K = k^2 = 25$  basis functions that are non-zero at the pseudo-coordinate  $\mathbf{u}_{ij}$  (dot).

$m$  with learnable coefficients  $a_0, \dots, a_m$  is

$$P(t) = \sum_{j=0}^m a_j t^j. \quad (3)$$

A *rational* function of degrees  $(m, n)$  is a quotient of two such polynomials,

$$R(t) = \frac{P(t)}{Q(t)} = \frac{\sum_{j=0}^m a_j t^j}{\sum_{j=0}^n b_j t^j}. \quad (4)$$

Training a free denominator is unstable, since it can become zero, making the evaluation undefined, but also being close to zero will lead to numerical overflow. We therefore use the *safe* Padé form (Molina et al., 2020), in which the denominator polynomial  $Q$  (without constant term) enters only through  $1 + |Q(\cdot)| \geq 1$ :

$$B(t) = \frac{P(t)}{1 + |Q(t)|} = \frac{\sum_{j=0}^m a_j t^j}{1 + \left| \sum_{j=1}^n b_j t^j \right|}, \quad (5)$$

which remains well defined and bounded for every value of the learnt coefficients  $a_j, b_j$ . Figure 3 contrasts the plain rational (4) with the safe form (5).

In practice we expand  $P$  and  $Q$  not in the monomial basis  $1, t, t^2, \dots$  but in Chebyshev polynomials of the first kind  $\phi_0, \phi_1, \dots$  ( $\phi_0=1$ ,  $\phi_1=t$ ,  $\phi_{j+1}=2t\phi_j - \phi_{j-1}$ ), i.e.  $P(t) = \sum_j a_j \phi_j(t)$ . This parametrization spans the same function class but is better conditioned on bounded domains. the map from monomial coefficients to polynomial values on an interval is exponentially ill-conditioned in the degree (Gautschi, 1979), so with monomials a small step in a high-order coefficient barely moves the function near  $t = 0$  but changes it violently near the boundary, giving gradients on wildly different scales. The Chebyshev polynomials, by contrast, are orthogonal and uniformly bounded,  $|\phi_j(t)| \leq 1$  on  $[-1, 1]$ , so every coefficient acts on a comparable scale and  $|a_j|$  directly bounds the contribution of the  $j$ -th term; Chebyshev expansions are the standard well-conditioned representation of polynomials on bounded intervals in numerical practice (Trefethen, 2019).

Below we extend this one-dimensional basis function to multiple dimensions in two different ways.

**Product rational basis.** Each of the  $K$  basis functions factorises over the coordinate dimensions into univariate safe Padé functions (5),

$$B_{\mathbf{p}}(\mathbf{u}) = \prod_{d=1}^D \frac{P_{d,p}(u_d)}{1 + |Q_{d,p}(u_d)|}, \quad (6)$$

where every factor has its own learnable polynomials  $P_{d,p}$  and  $Q_{d,p}$ .

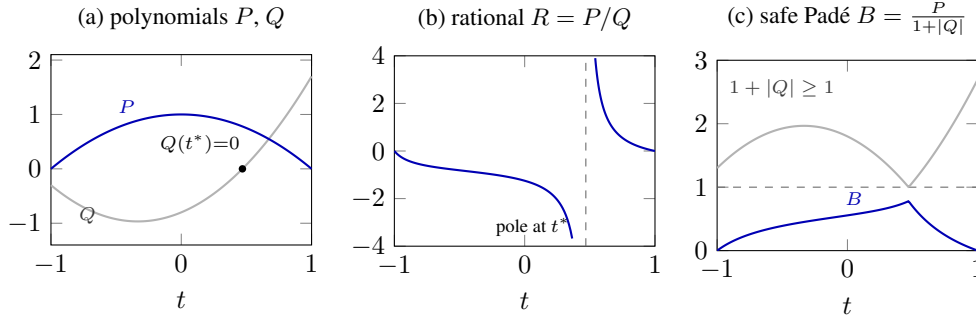


Figure 3: From polynomials to safe Padé basis functions, Eqs. (3)–(5), with the same numerator  $P$  and denominator  $Q$  in all three panels. **(a)** Two polynomials with learnable coefficients;  $Q$  has a root  $t^*$  inside the domain. **(b)** The plain rational function  $R = P/Q$  diverges at  $t^*$ , and any root of  $Q$  can move into the domain during training. **(c)** In the safe form the denominator  $1 + |Q|$  (gray) never drops below one (dashed), so  $B$  stays bounded and well defined for every setting of the coefficients; where  $Q$  vanishes,  $B$  simply equals  $P$ .

**Multivariate rational basis.** Each basis function  $B_p: [0, 1]^D \rightarrow \mathbb{R}$  is a single non-separable rational function of all coordinates jointly:

$$B_p(\mathbf{u}) = \frac{P_p(\mathbf{u})}{1 + |Q_p(\mathbf{u})|}, \quad P_p(\mathbf{u}) = \sum_{|\alpha| \leq m} a_{p,\alpha} u_1^{\alpha_1} \cdots u_D^{\alpha_D}, \quad Q_p(\mathbf{u}) = \sum_{1 \leq |\alpha| \leq n} b_{p,\alpha} u_1^{\alpha_1} \cdots u_D^{\alpha_D}, \quad (7)$$

with multi-indices  $\alpha \in \mathbb{N}_0^D$  of total degree  $|\alpha| \leq m$  and  $|\alpha| \leq n$ . As in the univariate case, the implementation uses the Chebyshev parametrization. Each monomial  $u_1^{\alpha_1} \cdots u_D^{\alpha_D}$  is replaced by the product  $\phi_{\alpha_1}(t_1) \cdots \phi_{\alpha_D}(t_D)$ , spanning the same function class in better-conditioned form.

**Original SplineCNN and B-spline basis.** The original SplineCNN Fey et al. (2018) uses the operator (2), but chooses the basis functions  $B_p$  as B-splines, see Figure 2. This construction yields spatial localisation and a strong geometric inductive bias. However, in every coordinate at most 2 B-splines are active, which means in larger dimensions that all but  $2^D$  of the  $\Theta_p$  receive zero gradient. This results in sparse, uneven updates and dense evaluation wasted on zeros. The basis size  $K = k^D$  grows exponentially with the coordinate dimension  $D$ .

**Initialization.** Randomly initializing the basis functions works consistently worse. We observe  $-0.7$  to  $-5.8$  drops in accuracy, see the ablation in our experiments. Since we use small random numbers for initialization the resulting basis functions are approximately flat. First, spatial localization would need to be completely learned from scratch. Second, near-identical basis activations give the different  $\Theta_p$  strongly correlated gradients. Therefore, we propose two initializations that allow for spatial localization from the beginning.

*Spline fit.* Fit each rational function to its B-spline by least squares so the layer starts as approximately SplineConv. We perform a stage-wise optimization to fit our learnable basis functions as follows:

1. We fit the numerator  $P$  with a closed-form least squares fit.
2. Next, we refine both  $P$  and  $Q$  with Adam (lr 0.01 for 500 steps).
3. Last, for further numerical accuracy we use L-BFGS for 100 iterations on float64.

We evaluate squared loss on a fixed grid of points, using 256 for 1D,  $32^2$  for 2D and  $16^3$  on 3D. We use a small values for the denominator  $Q$  since the gradient of a constant zero  $Q$  would vanish. The fit is cached, so stacked layers share the same fit. The spline fit can be performed whenever the number of basis functions  $K = k^D$  for some  $k \in \mathbb{N}$ .

*PCA of splines.* When  $K \neq k^D$ , we cannot fit to the splines. Instead, we fit the  $K$  functions to the  $K$  leading principal components of the  $k^D$  spline basis. In detail, this is done as follows: We sample points on the same grid as above. We collect the values in a matrix  $A$ , compute its SVD and the

the  $K$  leading left singular vectors  $u_1, \dots, u_K$ . We rescale each  $u_i$  to unit maximum and fix to a deterministic sign. Last, we fit each  $u_i$  with the same procedure as above for the spline fit.

*Variance-preserving gain.* The initialization of the kernel weights is calibrated for the spline basis. The entries of  $\Theta_p$  are drawn i.i.d. zero-mean with variance proportional to  $1/(KC_{\text{in}})$ . Thus, the message  $\sum_p B_p(\mathbf{u}) \Theta_p \mathbf{x}_j$  along an edge has variance proportional to the basis energy  $\sum_p B_p(\mathbf{u})^2$ . For partition-of-unity basis like the spline the variance remains. However, the PCA-based initialization can result in larger activations and, without correction, to inflated output scale.

Adapting a technique from KAT (Yang & Wang, 2025), we therefore rescale the kernel weights by  $\sqrt{\alpha}$  after basis initialization. The gain compares the energy of the B-spline basis being replaced with that of the initialized rational basis,

$$\alpha = \mathbb{E}_{\mathbf{u}} \left[ \sum_{\mathbf{p}} N_{\mathbf{p}}(\mathbf{u})^2 \right] / \mathbb{E}_{\mathbf{u}} \left[ \sum_p B_p(\mathbf{u})^2 \right], \quad (8)$$

where  $N_{\mathbf{p}}$  are the  $k^D$  tensor-product B-spline hats of Figure 2,  $B_p$  are our  $K$  rational basis functions as returned by the spline or PCA fit above, and both expectations are over uniform  $\mathbf{u}$ , evaluated on the fit grid. The initial message variance then matches that of SplineConv with the same  $K$ .

**Efficient implementation.** A naive implementation of RBF-GNN materialises the Chebyshev feature tensor before contracting it with the coefficients. More efficiently, we provide a triton kernel that never materializes this intermediate tensor that directly produces only  $K$  basis values from the  $D$  pseudo-coordinates. The polynomial sums are efficiently computed by a Horner-like scheme. The resulting kernel runs about  $15x$  times faster than the naive PyTorch implementation and  $3x$  faster than a compiled variant and consumes less memory. Training and inference with the Triton kernel for RBF-GNN is slightly faster overall than the PyG-implementation of SplineCNN.

## 4 EXPERIMENTS

We evaluate RBF-GNN on eight benchmarks from five tasks, with pseudo-coordinates of two to six dimensions. In each experiment, RBF-GNN replaces the SplineConv layers of the host network. SplineCNN is the original B-spline operator with  $K = k^D$  basis functions. Hat init and PCA+vp

Table 1: Semantic keypoint matching on PascalVOC and SPair-71k. Accuracy (%) follows the protocol of each pipeline and is the mean  $\pm$  std over 5 seeds, and **best**, **second**, and **third** mark the three highest means per column. Parameter counts cover the whole model for DGMC and only the graph-convolution layers for NMT. DGMC on SPair-71k reports the validation-selected checkpoint, and the other columns report the best test epoch.

| Method                                                   | DGMC (Fey et al., 2020) |                                    |                                    | NMT (Pourhadi & Swoboda, 2026) |                                    |                                    |
|----------------------------------------------------------|-------------------------|------------------------------------|------------------------------------|--------------------------------|------------------------------------|------------------------------------|
|                                                          | Params $\downarrow$     | PascalVOC $\uparrow$               | SPair-71k $\uparrow$               | Params $\downarrow$            | PascalVOC $\uparrow$               | SPair-71k $\uparrow$               |
| <i>Spline baselines</i>                                  |                         |                                    |                                    |                                |                                    |                                    |
| SplineCNN ( $K=25$ )                                     | 9.50M                   | 73.00 $\pm$ 0.40                   | 76.18 $\pm$ 0.64                   | 28.17M                         | <b>83.10 <math>\pm</math> 0.36</b> | <b>82.10 <math>\pm</math> 0.11</b> |
| SplineCNN ( $K=9$ )                                      | 3.74M                   | 72.78 $\pm$ 0.40                   | 76.04 $\pm$ 0.43                   | 10.84M                         | 82.66 $\pm$ 0.30                   | 81.54 $\pm$ 0.31                   |
| SplineCNN ( $K=4$ )                                      | 1.93M                   | 71.48 $\pm$ 0.40                   | 71.92 $\pm$ 0.38                   | 5.42M                          | 80.10 $\pm$ 0.46                   | 77.79 $\pm$ 0.40                   |
| <i>MLP learned-basis control</i>                         |                         |                                    |                                    |                                |                                    |                                    |
| MLP basis ( $K=4$ )                                      | 1.94M                   | 72.76 $\pm$ 0.27                   | 73.74 $\pm$ 0.71                   | 5.42M                          | 81.90 $\pm$ 0.27                   | 80.40 $\pm$ 0.57                   |
| MLP basis ( $K=9$ )                                      | 3.74M                   | 72.04 $\pm$ 0.73                   | 74.34 $\pm$ 0.62                   | 10.84M                         | 82.12 $\pm$ 0.51                   | 81.05 $\pm$ 0.32                   |
| <i>RBF-GNN with a product basis</i>                      |                         |                                    |                                    |                                |                                    |                                    |
| <b>RBF-GNN</b> ( $K=25$ )                                | 9.50M                   | 72.88 $\pm$ 0.48                   | 76.34 $\pm$ 0.42                   | 28.17M                         | <b>83.78 <math>\pm</math> 0.35</b> | <b>82.08 <math>\pm</math> 0.21</b> |
| <b>RBF-GNN</b> ( $K=9$ )                                 | 3.74M                   | 73.26 $\pm$ 0.18                   | 76.24 $\pm$ 0.73                   | 10.84M                         | 82.85 $\pm$ 0.42                   | 81.64 $\pm$ 0.35                   |
| <i>RBF-GNN with a multivariate basis</i>                 |                         |                                    |                                    |                                |                                    |                                    |
| <b>RBF-GNN</b> ( $K=9$ ), hat init                       | 3.74M                   | <b>74.12 <math>\pm</math> 0.48</b> | <b>77.68 <math>\pm</math> 0.35</b> | 10.84M                         | 83.08 $\pm$ 0.26                   | <b>82.11 <math>\pm</math> 0.35</b> |
| <b>RBF-GNN</b> ( $K=4$ ), PCA+vp                         | 1.94M                   | <b>73.78 <math>\pm</math> 0.41</b> | <b>77.44 <math>\pm</math> 0.36</b> | 5.42M                          | 82.42 $\pm$ 0.43                   | 81.52 $\pm$ 0.22                   |
| <b>RBF-GNN</b> ( $K=6$ ), PCA+vp                         | 2.66M                   | <b>74.06 <math>\pm</math> 0.37</b> | <b>77.80 <math>\pm</math> 0.66</b> | 7.59M                          | 82.95 $\pm$ 0.42                   | 81.71 $\pm$ 0.18                   |
| <b>RBF-GNN</b> ( $K=9$ ), PCA+vp                         | 3.74M                   | 73.72 $\pm$ 0.19                   | <b>77.50 <math>\pm</math> 0.27</b> | 10.84M                         | 82.88 $\pm$ 0.38                   | 81.59 $\pm$ 0.14                   |
| <i>RBF-GNN with a multivariate basis, degree control</i> |                         |                                    |                                    |                                |                                    |                                    |
| <b>RBF-GNN</b> ( $K=25$ ), degrees (5, 4)                | 9.51M                   | 73.08 $\pm$ 0.25                   | 76.26 $\pm$ 0.23                   | 28.17M                         | <b>83.35 <math>\pm</math> 0.38</b> | 82.01 $\pm$ 0.21                   |

Table 2: Accuracy (%) for shape correspondence on FAUST.

| Method                                     | Params ↓ | Final ↑          |
|--------------------------------------------|----------|------------------|
| <i>Spline baselines</i>                    |          |                  |
| <i>SplineCNN</i> ( $K=125$ ), published    | 4.10M    | 99.20            |
| <i>SplineCNN</i> ( $K=125$ ), mean + clip  | 4.11M    | 98.71 $\pm$ 0.08 |
| <i>SplineCNN</i> ( $K=125$ ), add + clip   | 4.11M    | 97.23 $\pm$ 0.51 |
| <i>SplineCNN</i> ( $K=125$ ), add, no clip | 4.11M    | 29.1 $\pm$ 50.3  |
| <i>SplineCNN</i> ( $K=27$ )                | 2.30M    | 83.6 $\pm$ 15.6  |
| <i>SplineCNN</i> ( $K=8$ )                 | 1.95M    | 3.4 $\pm$ 3.3    |
| <i>MLP learned-basis control</i>           |          |                  |
| MLP basis ( $K=8$ )                        | 1.96M    | 98.21 $\pm$ 1.11 |
| <i>RBF-GNN with a product basis</i>        |          |                  |
| <b>RBF-GNN</b> ( $K=27$ )                  | 2.30M    | 99.24 $\pm$ 0.08 |
| <i>RBF-GNN with a multivariate basis</i>   |          |                  |
| <b>RBF-GNN</b> ( $K=4$ )                   | 1.89M    | 99.38 $\pm$ 0.09 |
| <b>RBF-GNN</b> ( $K=4$ ), mean aggr.       | 1.89M    | 98.93 $\pm$ 0.10 |
| <b>RBF-GNN</b> ( $K=8$ )                   | 1.97M    | 99.41 $\pm$ 0.04 |
| <b>RBF-GNN</b> ( $K=16$ )                  | 2.13M    | 70.4 $\pm$ 25.2  |

Table 3: Instance-average mIoU (%) for part segmentation on ShapeNet-Part with 6-D position and normal pseudo-coordinates.

| Method                                   | Params ↓ | Final mIoU ↑     |
|------------------------------------------|----------|------------------|
| <i>Spline baselines</i>                  |          |                  |
| <i>SplineCNN</i> ( $K=729$ )             | 1.60M    | 65.12 $\pm$ 0.41 |
| <i>SplineCNN</i> ( $K=64$ )              | 177.91k  | 60.02 $\pm$ 0.23 |
| <i>MLP learned-basis control</i>         |          |                  |
| MLP basis ( $K=8$ )                      | 60.75k   | 62.80 $\pm$ 0.45 |
| MLP basis ( $K=32$ )                     | 116.88k  | 63.81 $\pm$ 0.46 |
| <i>RBF-GNN with a multivariate basis</i> |          |                  |
| <b>RBF-GNN</b> ( $K=8$ )                 | 64.87k   | 61.88 $\pm$ 0.38 |
| <b>RBF-GNN</b> ( $K=32$ )                | 137.43k  | 64.86 $\pm$ 0.18 |

denote the spline fit and the variance-preserving PCA initialization of Section 3. The MLP basis control produces the  $K$  basis functions with a two-layer MLP of the pseudo-coordinates (Schütt et al., 2017; Wu et al., 2019), and the PointNet control replaces the basis by a PointNet-style layer on the neighbor features and pseudo-coordinates. Results are means and standard deviations over three seeds unless stated otherwise.

#### 4.1 SEMANTIC KEYPOINT MATCHING

We evaluate semantic keypoint matching in two host pipelines, DGMC (Fey et al., 2020) and the Normalized Matching Transformer (NMT) (Pourhadi & Swoboda, 2026), which both use a continuous-kernel graph convolution for geometric feature refinement. In each pipeline we replace only the SplineConv layers and keep the rest of the architecture, the backbone and the training procedure unchanged. Table 1 reports all configurations for both pipelines on PascalVOC-Keypoints (Everingham et al., 2010; Bourdev & Malik, 2009) and SPair-71k (Min et al., 2019).

In every column of Table 1, the most accurate configuration uses a rational basis, though not always the same variant. The multivariate basis is best under DGMC on both datasets and, by 0.01, under NMT on SPair-71k, and the product basis is best under NMT on PascalVOC with 83.78. The margin over the best spline baseline decreases from +1.12 and +1.62 under DGMC to +0.68 and +0.01 under the stronger NMT pipeline, so the accuracy gain depends on the host pipeline. The parameter savings are more consistent. In three of the four columns, a rational basis matches or exceeds the widest spline grid with 2.6 to  $4.9\times$  fewer parameters, and under NMT on PascalVOC it is within 0.02 points of that grid with  $2.6\times$  fewer.

#### 4.2 SHAPE CORRESPONDENCE

Dense shape correspondence on FAUST (Bogo et al., 2014) assigns every vertex of a held-out human mesh to a template vertex. We follow the SplineCNN protocol (Fey et al., 2018), report exact-match accuracy and compare with the published SplineCNN result (Table 2). Small multivariate rational bases achieve the best and most stable results with substantially fewer parameters than SplineCNN.

#### 4.3 EVENT-BASED RECOGNITION

Both event-camera benchmarks use the seven-layer AEGNN recognition network (Schaefer et al., 2022), with one graph node per event and space-time pseudo-coordinates, and we replace only its SplineConv layers. We report test accuracy for multiclass object recognition on N-Caltech101 (Orchard et al., 2015), where we train with augmentation and a validation-based stopping criterion, and for real-world binary car-versus-background recognition on N-Cars (Sironi et al., 2018), where we

Table 4: Test accuracy (%) for object recognition on N-Caltech101 with AEGNN.

| Method                                   | Aggr. | Params ↓ | Final ↑          |
|------------------------------------------|-------|----------|------------------|
| <i>Spline baselines</i>                  |       |          |                  |
| SplineCNN ( $K=8$ )                      | mean  | 77.68k   | $47.35 \pm 0.15$ |
| SplineCNN ( $K=8$ )                      | max   | 77.68k   | $56.00 \pm 0.92$ |
| <i>PointNet learned-kernel control</i>   |       |          |                  |
| PointNet                                 | max   | 55.68k   | $54.99 \pm 0.84$ |
| <i>RBF-GNN with a multivariate basis</i> |       |          |                  |
| RBF-GNN ( $K=8$ )                        | mean  | 91.57k   | $55.05 \pm 0.58$ |
| RBF-GNN ( $K=8$ )                        | max   | 91.57k   | $57.55 \pm 0.42$ |

Table 5: Test accuracy (%) for car recognition on N-Cars with AEGNN.

| Method                                   | Aggr. | Params ↓ | Final ↑          |
|------------------------------------------|-------|----------|------------------|
| <i>Spline baselines</i>                  |       |          |                  |
| SplineCNN ( $K=8$ )                      | mean  | 26.99k   | $87.83 \pm 0.25$ |
| SplineCNN ( $K=8$ )                      | max   | 26.99k   | $89.81 \pm 0.27$ |
| <i>PointNet learned-kernel control</i>   |       |          |                  |
| PointNet                                 | max   | 4.99k    | $87.17 \pm 0.24$ |
| <i>RBF-GNN with a multivariate basis</i> |       |          |                  |
| RBF-GNN ( $K=4$ )                        | mean  | 21.10k   | $90.24 \pm 0.39$ |
| RBF-GNN ( $K=8$ )                        | mean  | 40.88k   | $90.49 \pm 0.05$ |
| RBF-GNN ( $K=8$ )                        | max   | 40.88k   | $91.00 \pm 0.43$ |

train for 30 epochs. On N-Caltech101, RBF-GNN performs best with max aggregation and also improves substantially over SplineCNN under matched mean aggregation (Table 4). On N-Cars, RBF-GNN is the most accurate model under both aggregation rules, its mean-aggregation variant outperforms the max-aggregation SplineCNN, and its smaller configuration is more accurate than SplineCNN with fewer parameters (Table 5).

#### 4.4 SUPERPIXEL GRAPH CLASSIFICATION

On MNIST (LeCun et al., 1998) superpixels, we follow the SplineCNN protocol (Fey et al., 2018) and use two continuous-kernel convolution layers on 2-D Cartesian pseudo-coordinates. On CIFAR-10 (Krizhevsky, 2009) superpixels from GNNBenchmarkDataset (Dwivedi et al., 2023), we use a 5-D bilateral pseudo-coordinate that concatenates the relative 2-D centroid position with the RGB features. We report test accuracy on both datasets. On MNIST, RBF-GNN achieves the three highest accuracies and improves over SplineCNN at both matched and reduced basis sizes (Table 6). On CIFAR-10, RBF-GNN attains the best accuracy while using far fewer parameters than the wider spline grid, and its smaller configurations remain competitive (Table 7).

#### 4.5 PART SEGMENTATION

For point-cloud part segmentation on ShapeNet-Part (Yi et al., 2016), we use a fixed subset of 4,000 training and 1,333 test shapes. Relative positions and surface normals define 6-D pseudo-coordinates on a 16-nearest-neighbor graph, and we report instance-average mIoU. RBF-GNN nearly matches the large spline grid with an order-of-magnitude smaller model and outperforms the equal-width MLP control (Table 3). At the smallest width the MLP control is more accurate, so the advantage of RBF-GNN on this benchmark is parameter efficiency at larger widths, not a gain at every width.

#### 4.6 ABLATION STUDY

**Basis size.** For the PCA+vp rows of Table 1, accuracy rises sharply from  $K=1$  to  $K=4$  in all four columns, peaks at  $K=6$  under DGMC and changes little beyond it under NMT. At matched  $K$ , the advantage over SplineCNN is concentrated at small bases. It ranges from +0.42 to +5.52 for  $K \in \{4, 9\}$ , and at  $K=25$  it is negligible, between  $-0.02$  and  $+0.16$ , except under NMT on PascalVOC with +0.68. On FAUST, the multivariate basis degrades sharply, with high variance across seeds, at  $K=16$  and  $K=27$  (Table 2), which points to a fragile PCA initialization rather than insufficient capacity. At 5-D and 6-D, in contrast, RBF-GNN improves up to  $K=32$  (Tables 7 and 3).

**Basis form and polynomial degree.** At  $K=9$ , the multivariate basis with hat init outperforms the product basis in all four columns of Table 1, but with higher polynomial degrees, (8, 6) instead of (5, 4). In the degree control, where both forms use (5, 4) at  $K=25$ , they differ by at most 0.43 points. On FAUST, the product basis remains stable at  $K=27$ , whereas the multivariate basis degrades sharply (Table 2).

Table 6: Test accuracy (%) for MNIST superpixel classification.

| Method                                   | Params ↓ | Final ↑          |
|------------------------------------------|----------|------------------|
| <i>Spline baselines</i>                  |          |                  |
| SplineCNN ( $K=25$ )                     | 88.36k   | $97.41 \pm 0.13$ |
| SplineCNN ( $K=9$ )                      | 55.08k   | $96.49 \pm 0.12$ |
| SplineCNN ( $K=4$ )                      | 44.68k   | $94.83 \pm 0.11$ |
| <i>MLP learned-basis control</i>         |          |                  |
| MLP basis ( $K=4$ )                      | 45.59k   | $97.23 \pm 0.11$ |
| <i>RBF-GNN with a multivariate basis</i> |          |                  |
| RBF-GNN ( $K=4$ ), PCA+vp                | 45.26k   | $97.79 \pm 0.08$ |
| RBF-GNN ( $K=9$ ), PCA+vp                | 56.38k   | $98.00 \pm 0.16$ |
| RBF-GNN ( $K=9$ ), hat init              | 56.38k   | $97.87 \pm 0.06$ |

Table 7: Test accuracy (%) with 5-D bilateral pseudo-coordinates on CIFAR-10 superpixels.

| Method                                   | Params ↓ | Final ↑          |
|------------------------------------------|----------|------------------|
| <i>Spline baselines</i>                  |          |                  |
| SplineCNN ( $K=243$ )                    | 557.42k  | $58.73 \pm 0.26$ |
| SplineCNN ( $K=32$ )                     | 105.03k  | $57.55 \pm 0.34$ |
| <i>MLP learned-basis control</i>         |          |                  |
| MLP basis ( $K=8$ )                      | 55.39k   | $59.67 \pm 0.32$ |
| MLP basis ( $K=32$ )                     | 109.96k  | $59.02 \pm 0.10$ |
| <i>RBF-GNN with a multivariate basis</i> |          |                  |
| RBF-GNN ( $K=4$ )                        | 46.45k   | $57.58 \pm 0.23$ |
| RBF-GNN ( $K=8$ )                        | 56.47k   | $58.97 \pm 0.16$ |
| RBF-GNN ( $K=32$ )                       | 116.62k  | $59.87 \pm 0.40$ |

**Initialization.** The tables compare the two initializations only at  $K=9$ , where hat init outperforms PCA+vp by 0.18 to 0.52 points in all four columns of Table 1 and is 0.13 points worse on MNIST (Table 6). The two therefore perform similarly, and PCA+vp is required whenever  $K \neq k^D$ .

**Aggregation.** On FAUST, SplineCNN is most accurate with mean aggregation and unstable with add aggregation unless gradients are clipped, whereas RBF-GNN at  $K=4$  is more accurate with add than with mean (Table 2). On both event-camera benchmarks, max aggregation outperforms mean aggregation for SplineCNN and RBF-GNN alike, and since RBF-GNN is more accurate under both rules, its gain is not explained by the choice of aggregation alone (Tables 4 and 5).

**Structured and learned bases.** In Table 1, the best rational basis in each column exceeds the best MLP control by +1.36, +3.46, +1.66 and +1.06 points. Unlike the margin over SplineCNN, this margin does not decrease systematically from DGMC to NMT, which indicates that the rational form itself matters and not only the fact that the basis is learned. On MNIST, the advantage over the equal-width MLP control likewise supports the rational parameterization, whereas on CIFAR-10 and ShapeNet-Part the MLP control is more accurate at the smallest shared width. On both event-camera benchmarks, the PointNet control uses fewer parameters than RBF-GNN but is less accurate than its best configuration.

## AI USE STATEMENT

In this work, we used generative AI tools to assist in implementing, refactoring and debugging the experimental code. We have not used generative AI tools to generate synthetic data, to formulate mathematical claims or to assist with translation, and the writing of proofs as well as qualitative and thematic data analysis are not applicable to this work. Additionally, we used generative AI tools to polish the language and improve the readability of the manuscript. We have reviewed all AI-assisted work. AI-assisted code was verified and tested for correctness by the authors, and the reported results were checked against the logs of the corresponding runs. The authors take full responsibility for the integrity, accuracy, and entire content of the submission.

## ETHICS STATEMENT

All authors have read and adhere to the ICLR Code of Ethics. This work proposes a basis for continuous-kernel graph convolutions and evaluates it on established benchmarks that are available to the research community. We collected no new data and conducted no experiments with human participants. Some of the benchmarks contain images or three-dimensional scans of people. We used them only as distributed by their creators and in accordance with their licences and terms of use, several of which restrict use to non-commercial research, and we made no attempt to identify any individual. The contribution is methodological and not tied to a particular application, so we do not foresee harmful uses beyond those that apply to graph learning methods in general.

## REPRODUCIBILITY STATEMENT

An anonymised version of our code is provided as supplementary material and will be released publicly under an open-source licence upon acceptance. Section 3 specifies the rational basis, both initialisation procedures and the variance-preserving rescaling, and Section 4 states the task, the metric and the number of random seeds for every benchmark. All reported results are means and standard deviations over independent runs with different seeds. Apart from the replaced basis, every host architecture keeps its published configuration unless Section 4 states otherwise.

## REFERENCES

- Randall Balestriero, Romain Cosentino, Hervé Glotin, and Richard Baraniuk. Spline filters for end-to-end deep learning. In *International Conference on Machine Learning (ICML)*, pp. 364–373, 2018.
- Federica Bogo, Javier Romero, Matthew Loper, and Michael J. Black. FAUST: Dataset and evaluation for 3D mesh registration. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3794–3801, 2014. doi: 10.1109/CVPR.2014.491.
- Nicolas Boulle, Yuji Nakatsukasa, and Alex Townsend. Rational neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 14243–14253, 2020.
- Lubomir Bourdev and Jitendra Malik. Poselets: Body part detectors trained using 3D human pose annotations. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 1365–1372, 2009. doi: 10.1109/ICCV.2009.5459303.
- Vijay Prakash Dwivedi, Chaitanya K. Joshi, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. Benchmarking graph neural networks. *Journal of Machine Learning Research*, 24(43):1–48, 2023.
- Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The Pascal Visual Object Classes (VOC) Challenge. *International Journal of Computer Vision*, 88(2):303–338, 2010. doi: 10.1007/s11263-009-0275-4.
- Matthias Fey, Jan Eric Lenssen, Frank Weichert, and Heinrich Müller. SplineCNN: Fast geometric deep learning with continuous B-spline kernels. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 869–877, 2018.
- Matthias Fey, Jan Eric Lenssen, Christopher Morris, Jonathan Masci, and Nils M. Kriege. Deep graph matching consensus. In *International Conference on Learning Representations (ICLR)*, 2020.

- Walter Gautschi. The condition of polynomials in power form. *Mathematics of Computation*, 33 (145):343–352, 1979. doi: 10.1090/S0025-5718-1979-0514830-6.
- Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning (ICML)*, pp. 1263–1272, 2017.
- Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. KAN: Kolmogorov–Arnold networks. In *International Conference on Learning Representations (ICLR)*, 2025.
- Liheng Ma, Soumyasundar Pal, Yitian Zhang, Jiaming Zhou, Yingxue Zhang, and Mark Coates. CKGConv: General graph convolution with continuous kernels. In *International Conference on Machine Learning (ICML)*, pp. 33902–33924, 2024.
- Juhong Min, Jongmin Lee, Jean Ponce, and Minsu Cho. SPair-71k: A large-scale benchmark for semantic correspondence. *arXiv preprint arXiv:1908.10543*, 2019.
- Alejandro Molina, Patrick Schramowski, and Kristian Kersting. Padé activation units: End-to-end learning of flexible activation functions in deep networks. In *International Conference on Learning Representations (ICLR)*, 2020.
- Federico Monti, Davide Boscaini, Jonathan Masci, Emanuele Rodolà, Jan Svoboda, and Michael M. Bronstein. Geometric deep learning on graphs and manifolds using mixture model CNNs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5115–5124, 2017.
- Garrick Orchard, Ajinkya Jayawant, Gregory K. Cohen, and Nitish Thakor. Converting static image datasets to spiking neuromorphic datasets using saccades. *Frontiers in Neuroscience*, 9:437, 2015. doi: 10.3389/fnins.2015.00437.
- Abtin Pourhadi and Paul Swoboda. Normalized Matching Transformer. In *International Conference on Pattern Recognition (ICPR)*, Lecture Notes in Computer Science, pp. 339–353. Springer, 2026. doi: 10.1007/978-3-032-31583-0\_23.
- David W. Romero, Robert-Jan Bruintjes, Erik J. Bekkers, Jakub M. Tomczak, Mark Hoogendoorn, and Jan C. van Gemert. FlexConv: Continuous kernel convolutions with differentiable kernel sizes. In *International Conference on Learning Representations (ICLR)*, 2022a.
- David W. Romero, Anna Kuzina, Erik J. Bekkers, Jakub M. Tomczak, and Mark Hoogendoorn. CKConv: Continuous kernel convolution for sequential data. In *International Conference on Learning Representations (ICLR)*, 2022b.
- Anit Kumar Sahu, Rizal Fathony, Devin Willmott, and J. Zico Kolter. Multiplicative filter networks. In *International Conference on Learning Representations (ICLR)*, 2021.
- Simon Schaefer, Daniel Gehrig, and Davide Scaramuzza. AEGNN: Asynchronous event-based graph neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12371–12381, 2022.
- Kristof T. Schütt, Pieter-Jan Kindermans, Huziel E. Sauceda Felix, Stefan Chmiela, Alexandre Tkatchenko, and Klaus-Robert Müller. SchNet: A continuous-filter convolutional neural network for modeling quantum interactions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- Martin Simonovsky and Nikos Komodakis. Dynamic edge-conditioned filters in convolutional neural networks on graphs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3693–3702, 2017.

- Amos Sironi, Manuele Brambilla, Nicolas Bourdis, Xavier Lagorce, and Ryad Benosman. HATS: Histograms of averaged time surfaces for robust event-based object classification. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10211–10220, 2018.
- Vincent Sitzmann, Julien N. P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 7462–7473, 2020.
- Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 7537–7547, 2020.
- Hugues Thomas, Charles R. Qi, Jean-Emmanuel Deschaud, Beatriz Marcotegui, François Goulette, and Leonidas J. Guibas. KPConv: Flexible and deformable convolution for point clouds. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 6411–6420, 2019.
- Lloyd N. Trefethen. *Approximation Theory and Approximation Practice*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, extended edition, 2019. doi: 10.1137/1.9781611975949.
- Nitika Verma, Edmond Boyer, and Jakob Verbeek. FeaStNet: Feature-steered graph convolutions for 3D shape analysis. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2598–2606, 2018.
- Shenlong Wang, Simon Suo, Wei-Chiu Ma, Andrei Pokrovsky, and Raquel Urtasun. Deep parametric continuous convolutional neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2589–2597, 2018.
- Wenxuan Wu, Zhongang Qi, and Li Fuxin. PointConv: Deep convolutional networks on 3D point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9621–9630, 2019.
- Mutian Xu, Runyu Ding, Hengshuang Zhao, and Xiaojuan Qi. PAConv: Position adaptive convolution with dynamic kernel assembling on point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3173–3182, 2021.
- Yifan Xu, Tianqi Fan, Mingye Xu, Long Zeng, and Yu Qiao. SpiderCNN: Deep learning on point sets with parameterized convolutional filters. In *European Conference on Computer Vision (ECCV)*, pp. 87–102, 2018.
- Xingyi Yang and Xinchao Wang. Kolmogorov–Arnold transformer. In *International Conference on Learning Representations (ICLR)*, 2025.
- Li Yi, Vladimir G. Kim, Duygu Ceylan, I-Chao Shen, Mengyan Yan, Hao Su, Cewu Lu, Qixing Huang, and Leonidas J. Guibas. A scalable active framework for region annotation in 3D shape collections. *ACM Transactions on Graphics*, 35(6):210:1–210:12, 2016. doi: 10.1145/2980179.2980238.
- Hengshuang Zhao, Li Jiang, Jiaya Jia, Philip H. S. Torr, and Vladlen Koltun. Point transformer. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 16259–16268, 2021.